Procedural Icicle Project - Code Listing
Generated from project source files

Included files:
  playground/openGLPlayground/main.cpp
  playground/openGLPlayground/shaderClass.h
  playground/openGLPlayground/shaderClass.cpp
  playground/Resource Files/Shaders/default.vert
  playground/Resource Files/Shaders/default.frag
  playground/Resource Files/Shaders/background.vert
  playground/Resource Files/Shaders/background.frag

External dependencies referenced by this code are not included: ituGL, GLAD, GLFW, GLM, stb_image, and texture assets such as iceCave.jpg.

================================================================================
playground/openGLPlayground/main.cpp
================================================================================
```
   1: #include <ituGL/core/DeviceGL.h>
   2: #include <ituGL/application/Window.h>
   3: #include <ituGL/geometry/VertexBufferObject.h>
   4: #include <ituGL/geometry/VertexArrayObject.h>
   5: #include <ituGL/geometry/VertexAttribute.h>
   6: #include <ituGL/geometry/ElementBufferObject.h>
   7: #include <iostream>
   8: #include <exception>
   9: #include <stdexcept>
  10: #include <cmath>
  11: #include <filesystem>
  12: #include <span>
  13: #include <string>
  14: #include "shaderClass.h"
  15: #include <glm/glm.hpp>
  16: #include <glm/gtc/matrix_transform.hpp>
  17: #include <glm/gtc/type_ptr.hpp>
  18: #include <vector>
  19: #include <random>
  20: #include <stb_image.h>
  21:
  22: // settings
  23: const unsigned int SCR_WIDTH = 1920;
  24: const unsigned int SCR_HEIGHT = 1080;
  25: const unsigned int FLOATS_PER_VERTEX = 6;
  26: const float PI = 3.1415926f;
  27: bool OVERLAPS = false;
  28:
  29: void processInput(GLFWwindow* window, float& cameraZ ,float& cameraVelocityZ);
  30:
  31: struct MeshData
  32: {
  33:     std::vector<float> vertices;
  34:     std::vector<unsigned int> indices;
  35: };
  36:
  37: struct IcicleSettings
  38: {
  39:     int segments = 24;
  40:     int count = 6000;
  41:     int maxPlacementAttempts = 6000;
```

```
  42:
  43:     float minRadius = 0.05f;
  44:     float maxRadius = 0.20f;
  45:
  46:     float topY = 0.5f;
  47:     float minTipY = -0.8f;
  48:     float maxTipY = -0.1f;
  49:
  50:     float roofWidth = 7.0f;
  51:     float roofDepth = 7.0f;
  52:     float minDistance = 0.44f;
  53:
  54:     bool allowOverlaps = false;
  55: };
  56:
  57: struct BackgroundQuad
  58: {
  59:     GLuint vao = 0;
  60:     GLuint vbo = 0;
  61:     GLuint texture = 0;
  62: };
  63:
  64: unsigned int getNextVertexIndex(const MeshData& mesh)
  65: {
  66:     return static_cast<unsigned int>(mesh.vertices.size() / FLOATS_PER_VERTEX);
  67: }
  68:
  69: std::filesystem::path resolveTexturePath(const char* filename)
  70: {
  71:     const std::filesystem::path textureFile(filename);
  72:     std::filesystem::path current = std::filesystem::current_path();
  73:
  74:     for (int i = 0; i < 6; ++i)
  75:     {
  76:         const auto resourcePath = current / "playground" / "Resource Files" / "Textures" / textureFile;
  77:         if (std::filesystem::exists(resourcePath))
  78:         {
  79:             return resourcePath;
  80:         }
  81:
  82:         const auto localResourcePath = current / "Resource Files" / "Textures" / textureFile;
  83:         if (std::filesystem::exists(localResourcePath))
  84:         {
  85:             return localResourcePath;
  86:         }
  87:
  88:         if (!current.has_parent_path())
  89:         {
  90:             break;
  91:         }
  92:
  93:         current = current.parent_path();
  94:     }
  95:
  96:     throw std::runtime_error(std::string("Could not find texture file: ") + filename);
  97: }
  98:
  99: GLuint loadTexture(const char* filename)
```

```
{
    const std::filesystem::path texturePath = resolveTexturePath(filename);

    int width = 0;
    int height = 0;
    int channels = 0;

    stbi_set_flip_vertically_on_load(true);
    unsigned char* data = stbi_load(texturePath.string().c_str(), &width, &height, &channels, 0);

    if (!data)
    {
        throw std::runtime_error("Failed to load texture file: " + texturePath.string());
    }

    GLenum format = GL_RGB;
    if (channels == 1)
    {
        format = GL_RED;
    }
    else if (channels == 4)
    {
        format = GL_RGBA;
    }

    GLuint textureID = 0;
    glGenTextures(1, &textureID);
    glBindTexture(GL_TEXTURE_2D, textureID);

    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0, format, GL_UNSIGNED_BYTE, data);
    glGenerateMipmap(GL_TEXTURE_2D);

    stbi_image_free(data);

    return textureID;
}

std::vector<glm::vec2> generateIciclePositions(const IcicleSettings& settings, std::mt19937& rng)
{
    std::uniform_real_distribution<float> randomX(
        -settings.roofWidth + settings.maxRadius,
        settings.roofWidth - settings.maxRadius
    );
    std::uniform_real_distribution<float> randomZ(
        -settings.roofDepth + settings.maxRadius,
        settings.roofDepth - settings.maxRadius
    );

    std::vector<glm::vec2> positions;

    int attempts = 0;
    while (static_cast<int>(positions.size()) < settings.count && attempts < settings.maxPlacementAttempts)
    {
```

```
        attempts++;

        glm::vec2 candidate(randomX(rng), randomZ(rng));
        bool overlaps = false;

        for (const glm::vec2& existing : positions)
        {
            float distance = glm::length(candidate - existing);

            if (distance < settings.minDistance && !settings.allowOverlaps)
            {
                overlaps = true;
                break;
            }
        }

        if (!overlaps)
        {
            positions.push_back(candidate);
        }
    }

    return positions;
}

void appendIcicleMesh(MeshData& mesh, const glm::vec2& center, const IcicleSettings& settings, std::mt19937& rng)
{
    std::uniform_real_distribution<float> randomRadius(settings.minRadius, settings.maxRadius);
    std::uniform_real_distribution<float> randomTipY(settings.minTipY, settings.maxTipY);
    std::uniform_real_distribution<float> randomTipOffset(-0.06f, 0.06f);
    std::uniform_real_distribution<float> randomShape(0.75f, 1.25f);

    const float centerX = center.x;
    const float centerZ = center.y;
    const float icicleRadius = randomRadius(rng);
    const float tipY = randomTipY(rng);
    const float tipOffsetX = randomTipOffset(rng);
    const float tipOffsetZ = randomTipOffset(rng);

    unsigned int tipIndex = getNextVertexIndex(mesh);

    // Tip vertex. The small x/z offset makes each icicle lean slightly.
    // position                         // normal
    mesh.vertices.insert(mesh.vertices.end(), {
        centerX + tipOffsetX, tipY, centerZ + tipOffsetZ,
        0.0f, -1.0f, 0.0f
    });

    unsigned int ringStartIndex = getNextVertexIndex(mesh);

    // Top ring vertices
    for (int i = 0; i < settings.segments; i++)
    {
        float angle = (2.0f * PI * i ) / settings.segments;
        float shapeNoise = randomShape(rng);

        float localX = std::cos(angle) * icicleRadius * shapeNoise;
        float localZ = std::sin(angle) * icicleRadius * shapeNoise;
```

```

        float x = centerX + localX;
        float z = centerZ + localZ;
        float height = settings.topY - tipY;

        glm::vec3 normal = glm::normalize(glm::vec3(
            localX,
            icicleRadius / height,
            localZ
        ));

        mesh.vertices.insert(mesh.vertices.end(), {
           x, settings.topY, z,                  normal.x, normal.y, normal.z
        });
    }

    // Side triangles
    for (int i = 0; i < settings.segments; i++) {
        unsigned int current = ringStartIndex + i;
        unsigned int next = ringStartIndex + ((i+1) % settings.segments);

        mesh.indices.insert(mesh.indices.end(), {
            tipIndex, current, next
        });
    }
}

void appendRoofMesh(MeshData& mesh, const IcicleSettings& settings)
{
    // Ceiling
    unsigned int roofStartIndex = getNextVertexIndex(mesh);

        // position                                     // normal
    mesh.vertices.insert(mesh.vertices.end(), {
        -settings.roofWidth, settings.topY, -settings.roofDepth,       0.0f, -1.0f, 0.0f,
        settings.roofWidth, settings.topY, -settings.roofDepth,        0.0f, -1.0f, 0.0f,
        settings.roofWidth, settings.topY,  settings.roofDepth,        0.0f, -1.0f, 0.0f,
        -settings.roofWidth, settings.topY,  settings.roofDepth,       0.0f, -1.0f, 0.0f
    });

    mesh.indices.insert(mesh.indices.end(), {
        roofStartIndex + 0, roofStartIndex + 2, roofStartIndex + 1,
        roofStartIndex + 0, roofStartIndex + 3, roofStartIndex + 2
    });
}

BackgroundQuad createBackgroundQuad(GLuint texture)
{
    float backgroundVertices[] = {
        // position    // tex coords
        -1.0f,  1.0f,  0.0f, 1.0f,
        -1.0f, -1.0f,  0.0f, 0.0f,
         1.0f, -1.0f,  1.0f, 0.0f,

        -1.0f,  1.0f,  0.0f, 1.0f,
         1.0f, -1.0f,  1.0f, 0.0f,
         1.0f,  1.0f,  1.0f, 1.0f
    };
```

```
274:
275:     BackgroundQuad background;
276:     background.texture = texture;
277:
278:     glGenVertexArrays(1, &background.vao);
279:     glGenBuffers(1, &background.vbo);
280:
281:     glBindVertexArray(background.vao);
282:     glBindBuffer(GL_ARRAY_BUFFER, background.vbo);
283:     glBufferData(GL_ARRAY_BUFFER, sizeof(backgroundVertices), backgroundVertices, GL_STATIC_DRAW);
284:
285:     glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 4 * sizeof(float), (void*)0);
286:     glEnableVertexAttribArray(0);
287:
288:     glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 4 * sizeof(float), (void*)(2 * sizeof(float)));
289:     glEnableVertexAttribArray(1);
290:
291:     glBindBuffer(GL_ARRAY_BUFFER, 0);
292:     glBindVertexArray(0);
293:
294:     return background;
295: }
296:
297: void drawBackground(const BackgroundQuad& background, Shader& backgroundShader)
298: {
299:     glDisable(GL_DEPTH_TEST);
300:     backgroundShader.Activate();
301:     glActiveTexture(GL_TEXTURE0);
302:     glBindTexture(GL_TEXTURE_2D, background.texture);
303:     glBindVertexArray(background.vao);
304:     glDepthMask(GL_FALSE);
305:     glDrawArrays(GL_TRIANGLES, 0, 6);
306:     glDepthMask(GL_TRUE);
307:     glBindVertexArray(0);
308:     glEnable(GL_DEPTH_TEST);
309: }
310:
311: int main()
312: {
313:     try {
314:         // glfw: initialize and configure
315:         // ------------------------------
316:         DeviceGL deviceGL;
317:
318:         // glfw window creation
319:         // --------------------
320:         Window window(SCR_WIDTH, SCR_HEIGHT, "LearnOpenGL");
321:         if (!window.IsValid())
322:         {
323:             std::cout << "Failed to create GLFW window" << std::endl;
324:             return -1;
325:         }
326:
327:         // glad: load all OpenGL function pointers
328:         // ---------------------------------------
329:         deviceGL.SetCurrentWindow(window);
330:         if (!deviceGL.IsReady())
331:         {
```

```
332:             std::cout << "Failed to initialize GLAD" << std::endl;
333:             return -1;
334:         }
335:
336:         // build and compile our shader program
337:         // ------------------------------------
338:         Shader shaderProgram("default.vert", "default.frag");
339:         Shader backgroundShader("background.vert", "background.frag");
340:
341:         BackgroundQuad background = createBackgroundQuad(loadTexture("iceCave.jpg"));
342:
343:         backgroundShader.Activate();
344:         glUniform1i(glGetUniformLocation(backgroundShader.ID, "backgroundTexture"), 0);
345:
346:         // Vertex data for icicles and roof, generated once on the CPU.
347:         MeshData sceneMesh;
348:         IcicleSettings icicleSettings;
349:         icicleSettings.allowOverlaps = OVERLAPS;
350:         std::mt19937 rng(123);
351:
352:         std::vector<glm::vec2> iciclePositions = generateIciclePositions(icicleSettings, rng);
353:         for (const glm::vec2& center : iciclePositions)
354:         {
355:             appendIcicleMesh(sceneMesh, center, icicleSettings, rng);
356:         }
357:
358:         appendRoofMesh(sceneMesh, icicleSettings);
359:
360:         // Vertex Array Object (VAO) - Part 2
361:         // Stores pointers to VAOs and tells opengl where to find them
362:         VertexArrayObject vao;
363:         vao.Bind();
364:
365:         // Vertex buffer object (VBO) - Part 1 of graphics pipeline
366:         // An array of references to the vertex data
367:         VertexBufferObject vbo;
368:         vbo.Bind();
369:         vbo.AllocateData<float>(std::span(sceneMesh.vertices));
370:
371:         // Element Buffer Object (EBO) - Part 3
372:         // Stores indicies that describe which vertices to draw - Indexed drawing!
373:         ElementBufferObject ebo;
374:         ebo.Bind();
375:         ebo.AllocateData<unsigned int>(std::span(sceneMesh.indices));
376:
377:         // The position attribute is made of 3 floats: x, y, z
378:         VertexAttribute position(Data::Type::Float, 3);
379:         // Use shader location 0, read 3 floats (x,y,z) start at byte offset 0, jump one full vertex to get to the next position
380:         vao.SetAttribute(0, position, 0, FLOATS_PER_VERTEX * sizeof(float));
381:
382:         // The normal attribute is made of 3 floats: nx, ny, nz
383:         VertexAttribute normalV(Data::Type::Float, 3);
384:         // Use shader location 1, read 3 floats after the position, jump one full vertex to get to the next normal
385:         vao.SetAttribute(1, normalV, 3 * sizeof(float), FLOATS_PER_VERTEX * sizeof(float));
386:
387:         // note that this is allowed, the call to glVertexAttribPointer registered VBO as the vertex attribute's bound vertex buffer object so afterwards
    | we can safely unbind
388:         VertexBufferObject::Unbind();
```

```
389:
390:         // You can unbind the VAO afterwards so other VAO calls won't accidentally modify this VAO, but this rarely happens. Modifying other
391:         // VAOs requires a call to glBindVertexArray anyways so we generally don't unbind VAOs (nor VBOs) when it's not directly necessary.
392:         VertexArrayObject::Unbind();
393:
394:         // Now we can unbind the EBO as well
395:         ElementBufferObject::Unbind();
396:
397:         shaderProgram.Activate();
398:
399:         // Send light values to shaderProgram
400:         glUniform3f(glGetUniformLocation(shaderProgram.ID, "lightPos"), 0.0f, 2.0f, 10.0f);
401:         glUniform3f(glGetUniformLocation(shaderProgram.ID, "objectColor"), 0.45f, 0.85f, 1.0f);
402:         glUniform3f(glGetUniformLocation(shaderProgram.ID, "lightColor"), 1.0f, 1.0f, 1.0f);
403:
404:         glUniform1f(glGetUniformLocation(shaderProgram.ID, "ambientStrength"), 0.18f);
405:         glUniform1f(glGetUniformLocation(shaderProgram.ID, "specularStrength"), 0.9f);
406:         glUniform1f(glGetUniformLocation(shaderProgram.ID, "shininess"), 64.0f);
407:         glUniform1f(glGetUniformLocation(shaderProgram.ID, "rimStrength"), 0.45f);
408:         glUniform1f(glGetUniformLocation(shaderProgram.ID, "alpha"), 0.96f);
409:
410:         // Depth Buffer
411:         deviceGL.EnableFeature(GL_DEPTH_TEST);
412:
413:         // If alpha is 0.45, icicle contributes 45% and the background contributes 55%
414:         glEnable(GL_BLEND);
415:         glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
416:
417:         float cameraZ = -6.0f;
418:         float cameraX = 0.0f;
419:         float cameraY = 0.35f;
420:         float cameraVelocityZ = 0.0f;
421:
422:         // Render loop
423:         while (!window.ShouldClose())
424:         {
425:
426:             processInput(window.GetInternalWindow(), cameraZ, cameraVelocityZ);
427:
428:             deviceGL.Clear(true, Color(0.2f, 0.3f, 0.3f, 1.0f), true, 1.0f);
429:
430:             drawBackground(background, backgroundShader);
431:
432:             shaderProgram.Activate();
433:
434:             glm::vec3 cameraWorldPos(-cameraX, -cameraY, -cameraZ);
435:             glm::mat4 model = glm::mat4(1.0f);
436:             glm::mat4 view = glm::mat4(1.0f);
437:             glm::mat4 proj = glm::mat4(1.0f);
438:
439:             model = glm::rotate(
440:                 model,
441:                 glm::radians((float)glfwGetTime() * -1.0f),
442:                 glm::vec3(0.0f, 1.0f, 0.0f)
443:             );
444:
445:
446:             view = glm::translate(view, glm::vec3(cameraX, cameraY, cameraZ));
```

```
447:
448:             proj = glm::perspective(
449:                 glm::radians(45.0f),
450:                 (float)SCR_WIDTH / (float)SCR_HEIGHT,
451:                 0.1f,
452:                 100.0f
453:             );
454:
455:             int modelLoc = glGetUniformLocation(shaderProgram.ID, "model");
456:             int viewLoc = glGetUniformLocation(shaderProgram.ID, "view");
457:             int projLoc = glGetUniformLocation(shaderProgram.ID, "proj");
458:
459:             glUniform3f(glGetUniformLocation(shaderProgram.ID, "viewPos"), cameraWorldPos.x, cameraWorldPos.y, cameraWorldPos.z);
460:             glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
461:             glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
462:             glUniformMatrix4fv(projLoc, 1, GL_FALSE, glm::value_ptr(proj));
463:
464:             vao.Bind();
465:
466:             glDrawElements(
467:                 GL_TRIANGLES,
468:                 static_cast<GLsizei>(sceneMesh.indices.size()),
469:                 GL_UNSIGNED_INT,
470:                 0
471:             );
472:
473:             window.SwapBuffers();
474:             deviceGL.PollEvents();
475:         }
476:
477:         // glfw: terminate, clearing all previously allocated GLFW resources.
478:         // ------------------------------------------------------------------
479:         // This is now done in the destructor of DeviceGL
480:         return 0;
481:     }
482:     catch (const std::exception& exception)
483:     {
484:         std::cerr << exception.what() << std::endl;
485:         return -1;
486:     }
487: }
488:
489: void processInput(GLFWwindow* window, float& cameraZ ,float& cameraVelocityZ)
490: {
491:     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
492:         glfwSetWindowShouldClose(window, true);
493:
494:     if (glfwGetKey(window, GLFW_KEY_Q) == GLFW_PRESS) {
495:             glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
496:     }
497:
498:     if (glfwGetKey(window, GLFW_KEY_1) == GLFW_PRESS) {
499:         glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
500:     }
501:
502:     if (glfwGetKey(window, GLFW_KEY_2) == GLFW_PRESS) {
503:         glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
504:     }
```

```
505:
506:     // Move
507:     if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS) {
508:         cameraVelocityZ += 0.0000001f;
509:     }
510:
511:     if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS) {
512:         cameraVelocityZ -= 0.0000001f;
513:     }
514:
515:     if (glfwGetKey(window, GLFW_KEY_SPACE) == GLFW_PRESS) {
516:         cameraVelocityZ = 0.0f;
517:     }
518:
519:     cameraZ += cameraVelocityZ;
520: }
```

playground/openGLPlayground/shaderClass.h

```
  1: //
  2: // Created by ander on 4/10/2026.
  3: //
  4:
  5: #ifndef ITU_GRAPHICS_PROGRAMMING_SHADERCLASS_H
  6: #define ITU_GRAPHICS_PROGRAMMING_SHADERCLASS_H
  7:
  8: #endif //ITU_GRAPHICS_PROGRAMMING_SHADERCLASS_H
  9:
 10: #ifndef SHADER_CLASS_H
 11: #define SHADER_CLASS_H
 12:
 13: #include<glad/glad.h>
 14: #include<filesystem>
 15: #include<string>
 16: #include<fstream>
 17: #include<sstream>
 18: #include<iostream>
 19: #include<stdexcept>
 20:
 21: // Function that reads the shader text files
 22: std::string get_file_contents(const char* filename);
 23:
 24: class Shader
 25: {
 26:     public:
 27:         // refernce ID
 28:         GLuint ID;
 29:         // Constructor
 30:         Shader(const char* vertexFile, const char* fragmentFile);
 31:
 32:         void Activate();
 33:         void Deactivate();
 34: };
 35:
 36: #endif
```

playground/openGLPlayground/shaderClass.cpp

```
#include "shaderClass.h"

namespace
{
    std::filesystem::path resolve_shader_path(const char* filename)
    {
        const std::filesystem::path shaderFile(filename);
        std::filesystem::path current = std::filesystem::current_path();

        for (int i = 0; i < 6; ++i)
        {
            const auto directPath = current / shaderFile;
            if (std::filesystem::exists(directPath))
            {
                return directPath;
            }

            const auto resourcePath = current / "playground" / "Resource Files" / "Shaders" / shaderFile;
            if (std::filesystem::exists(resourcePath))
            {
                return resourcePath;
            }

            const auto localResourcePath = current / "Resource Files" / "Shaders" / shaderFile;
            if (std::filesystem::exists(localResourcePath))
            {
                return localResourcePath;
            }

            if (!current.has_parent_path())
            {
                break;
            }

            current = current.parent_path();
        }

        throw std::runtime_error(std::string("Could not find shader file: ") + filename);
    }
}

std::string get_file_contents(const char *filename) {
    const auto path = resolve_shader_path(filename);
    std::ifstream in(path, std::ios::binary);
    if (in) {
        std::string contents;
        in.seekg(0, std::ios::end);
        contents.resize(in.tellg());
        in.seekg(0, std::ios::beg);
        in.read(&contents[0], contents.size());
        in.close();
        return (contents);
    }
    throw std::runtime_error("Failed to read shader file: " + path.string());
}

```

```
Shader::Shader(const char *vertexFile, const char *fragmentFile) {
    // Read files
    std::string vertexCode = get_file_contents(vertexFile);
    std::string fragmentCode = get_file_contents(fragmentFile);

    // Convert into character arrays
    const char* vertexSource = vertexCode.c_str();
    const char* fragmentSource = fragmentCode.c_str();

        // Create shader object
    // GL_VERTEX_SHADER is passed as that is the type of shader we want to create
    unsigned int vertexShader = glCreateShader(GL_VERTEX_SHADER);
    // Attaching shader source code to the shader object and compiling the shader
    glShaderSource(vertexShader, 1, &vertexSource, NULL);
    glCompileShader(vertexShader);
    // check for shader compile errors
    int success;
    char infoLog[512];
    glGetShaderiv(vertexShader, GL_COMPILE_STATUS, &success);
    if (!success)
    {
        glGetShaderInfoLog(vertexShader, 512, NULL, infoLog);
        glDeleteShader(vertexShader);
        throw std::runtime_error(std::string("ERROR::SHADER::VERTEX::COMPILATION_FAILED\n") + infoLog);
    }

    // Shader 2: Fragment shader
    // Calculates color output of our pixels
    // For simplicity, now we just do one color, orange

    // Compiling fragment shader
    unsigned int fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
    glShaderSource(fragmentShader, 1, &fragmentSource, NULL);
    glCompileShader(fragmentShader);
    // check for shader compile errors
    glGetShaderiv(fragmentShader, GL_COMPILE_STATUS, &success);
    if (!success)
    {
        glGetShaderInfoLog(fragmentShader, 512, NULL, infoLog);
        glDeleteShader(vertexShader);
        glDeleteShader(fragmentShader);
        throw std::runtime_error(std::string("ERROR::SHADER::FRAGMENT::COMPILATION_FAILED\n") + infoLog);
    }

    // Linking our 2 shaders to a shader program
    /* Shader program:
     * - Final linked version of multiple shaders combined
     * - After linking shaders to the shader program, we activate the shader program when rendering objects
     * - Activated shader program's shaders are used during render calls
    */
    ID = glCreateProgram();
    glAttachShader(ID, vertexShader);
    glAttachShader(ID, fragmentShader);
    glLinkProgram(ID);
    // Every shader and rendering call after glUseProgram will now use this program object
    // check for linking errors
    glGetProgramiv(ID, GL_LINK_STATUS, &success);
    if (!success) {
```

```
115:         glGetProgramInfoLog(ID, 512, NULL, infoLog);
116:         glDeleteShader(vertexShader);
117:         glDeleteShader(fragmentShader);
118:         glDeleteProgram(ID);
119:         throw std::runtime_error(std::string("ERROR::SHADER::PROGRAM::LINKING_FAILED\n") + infoLog);
120:     }
121:     // Deleting the shader objects after linking to the program object since we dont need them anymore
122:     glDeleteShader(vertexShader);
123:     glDeleteShader(fragmentShader);
124: }
125:
126: void Shader::Activate() {
127:     glUseProgram(ID);
128: }
129:
130: void Shader::Deactivate() {
131:     glDeleteProgram(ID);
132: }
```

playground/Resource Files/Shaders/default.vert

```
  1: #version 330 core
  2:
  3: layout (location = 0) in vec3 aPos;
  4: layout (location = 1) in vec3 aNormal;
  5:
  6: out vec3 FragPos;
  7: out vec3 Normal;
  8:
  9: uniform mat4 model;
 10: uniform mat4 view;
 11: uniform mat4 proj;
 12:
 13: void main()
 14: {
 15:    // Convert vertex position from local object space to world space
 16:    FragPos = vec3(model * vec4(aPos, 1.0));
 17:
 18:    // Convert normal direction from local object space to world space
 19:    Normal = mat3(transpose(inverse(model))) * aNormal;
 20:
 21:    // Convert world-space position to clip space
 22:    gl_Position = proj * view * vec4(FragPos, 1.0);
 23: }
```

playground/Resource Files/Shaders/default.frag

```
  1: #version 330 core
  2:
  3: in vec3 FragPos;
  4: in vec3 Normal;
  5:
  6: uniform vec3 lightPos;
  7: uniform vec3 viewPos;
  8:
  9: uniform vec3 objectColor;
```

```
uniform vec3 lightColor;

uniform float ambientStrength;
uniform float specularStrength;
uniform float shininess = 64.0f;
uniform float rimStrength;
uniform float alpha;

out vec4 FragColor;

void main()
{
    vec3 norm = normalize(Normal);

    // Direction from fragment to light
    vec3 lightDir = normalize(lightPos - FragPos);

    // Direction from fragment to camera
    vec3 viewDir = normalize(viewPos - FragPos);

    // AMBIENT
    vec3 ambient = ambientStrength * lightColor;

    // DIFFUSE
    float diffuseAmount = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = diffuseAmount * lightColor;

    // SPECULAR - Blinn-Phong
    vec3 halfwayDir = normalize(lightDir + viewDir);
    float specularAmount = pow(max(dot(norm, halfwayDir), 0.0), shininess);
    vec3 specular = specularStrength * specularAmount * lightColor;

    // RIM / FRESNEL-ISH EDGE GLOW
    float rimAmount = 1.0 - max(dot(norm, viewDir), 0.0);
    rimAmount = pow(rimAmount, 2.0);
    vec3 rim = rimAmount * rimStrength * vec3(0.5, 0.85, 1.0);

    vec3 result = (ambient + diffuse) * objectColor + specular + rim;

    FragColor = vec4(result, alpha);
}
```

playground/Resource Files/Shaders/background.vert

```
#version 330 core

layout (location = 0) in vec2 aPos;
layout (location = 1) in vec2 aTexCoord;

out vec2 TexCoord;

void main()
{
    TexCoord = aTexCoord;
    gl_Position = vec4(aPos, 0.0, 1.0);
}
```

playground/Resource Files/Shaders/background.frag

```
   1: #version 330 core
   2:
   3: in vec2 TexCoord;
   4:
   5: uniform sampler2D backgroundTexture;
   6:
   7: out vec4 FragColor;
   8:
   9: void main()
  10: {
  11:     FragColor = texture(backgroundTexture, TexCoord);
  12: }
```